Data Analytics and Visualization

Piotr Jastrzębski

2026-03-21

Table of contents

1 Data Analysis and Visualization

The current version pertains to classes held in the 2025/26 academic year.

Some of the texts have been machine-translated into English. In case of any ambiguities, please get in touch via email.

Part I

Introduction

2 A bit of theory...

2.1 Rational thinking test

- If 5 machines produce 5 devices in 5 minutes, how long will it take 100 machines to make 100 devices?
- A patch of water lilies is growing on a pond. Every day the patch doubles in size. If it takes 48 days for the lilies to cover the entire pond, how many days does it take for them to cover half the pond?
- A baseball bat and a ball cost $1.10 together. The bat costs one dollar more than the ball. How much does the ball cost?

2.2 Data analysis

Data analysis is the process of examining, cleaning, transforming, and modeling data in order to discover useful information, draw conclusions, and support decision-making. It is a multi-stage process that includes:

- Collecting data from various sources
- Cleaning data by removing errors, missing values, and inconsistencies
- Exploring data to understand its structure and characteristics
- Transforming data into the appropriate format
- Applying statistical methods and machine learning algorithms
- Interpreting results in the context of a specific business or scientific problem

Data analysis is used in nearly every field, from business and finance to social sciences, medicine, and scientific research. The goal of data analysis is to transform raw data into knowledge that can be used to make better decisions.

2.3 Data visualization

Data visualization is the graphical representation of information and data. It uses visual elements such as charts, maps, and dashboards to present relationships between data in a way that is easy to understand and interpret. Good data visualization:

- Presents complex information in an accessible and intuitive way
- Reveals patterns, trends, and outliers that may be difficult to notice in raw data
- Supports the data analysis process by enabling quick review of large datasets
- Facilitates communication of analysis results to various audiences, including non-technical individuals
- Helps tell stories contained in data (data storytelling)

The most popular types of data visualization include bar charts, line charts, pie charts, heatmaps, hierarchical trees, word clouds, and interactive dashboards. The choice of the appropriate visualization form depends on the type of data, the purpose of the presentation, and the target audience.

Data visualization is a key element of the data analysis process because it allows for quick drawing of conclusions and making decisions based on data. It is a bridge between complex data and human understanding.

2.4 Data analysis - basic concepts

2.4.1 Modern meanings of the word "statistics":

- a set of numerical data showing the development of processes and phenomena, e.g. population statistics.
- all activities related to collecting and processing numerical data, e.g. statistics on a certain problem conducted by the Central Statistical Office.
- numerical characteristics, e.g. sample statistics such as arithmetic mean, standard deviation, etc.
- a scientific discipline - the science of methods for studying mass phenomena.

2.4.2 "Massiveness"

Mass phenomena/processes - a large number of units are subject to study. They are divided into:

- economic (e.g. production, consumption, services, advertising),
- social (e.g. traffic accidents, political views),
- demographic (e.g. births, aging, migrations).

2.4.3 Division of statistics

Statistics - a scientific discipline - division:

- descriptive statistics - deals with matters related to collecting, presenting, analyzing, and interpreting numerical data. Observation covers the entire population under study.
- mathematical statistics - generalization of the results of studying a part of the population (sample) to the entire population.

2.4.4 Population

Statistical population: a set of objects subject to statistical study. It consists of units that are similar to each other, logically related, but not identical. They share certain common features and certain properties that allow them to be differentiated.

- examples:
  - study of the height of Polish citizens - inhabitants of Poland
  - level of education in schools of the Warmian-Masurian Voivodeship - schools of the Warmian-Masurian Voivodeship.
- division:
  - general population - covers the entirety,
  - sample population (sample) - covers a part of the population.

2.4.5 Statistical unit

Statistical unit: each element of a statistical population.

- examples:
  - UWM students - a UWM student
  - inhabitants of Poland - every person living in Poland
  - machines produced in a factory - every machine

2.4.6 Statistical features

Statistical features

- properties characterizing statistical units in a given statistical population.
- they are divided into constant and variable.

Constant features

- properties that are common to all units of a given statistical population.
- division:
  - substantive - who or what is the subject of the statistical study,
  - temporal - when the study was conducted or what time period the study covers,
  - spatial - what territory (place or area) the study concerns.
- example: students of the Faculty of Mathematics and Computer Science (WMiI) at UWM in Olsztyn in the academic year 2017/2018:
  - substantive feature: possession of a student ID card,
  - temporal feature - students studying in the academic year 2017/2018,
  - spatial feature - location: WMiI UWM in Olsztyn.

Variable features

- properties that differentiate statistical units in a given population.
- example: UWM students - variable features: age, sex, type of secondary school completed, eye color, height.

Important:

- only variable features are subject to observation,
- a constant feature in one population may be a variable feature in another population.

Example: UWM students have an ID card issued by UWM. Students of all universities in Poland have ID cards issued by different institutions.

Division of variable features:

- measurable (quantitative) features - can be expressed as a number with a specific unit of measurement.
- non-measurable (qualitative) features - described verbally, representing certain categories.

Example: student population. Measurable features: age, weight, height, number of absences. Non-measurable features: sex, eye color, field of study.

For practical reasons, numerical codes are often assigned to non-measurable features. However, they should not be confused with measurable features. E.g. 1 - primary education, 2 - vocational education, etc.

Division of measurable features:

- continuous - can take any value within a defined interval, e.g. height, age, apartment area.
- discrete - can take specific (discrete) numerical values without intermediate values, e.g. number of people in a household, number of employees in a given company.

Discrete features usually have integer values, although this is not always required, e.g. the number of full-time positions in a company (including part-time positions).

2.4.7 Scales

Measurement scale

- a system that allows the results of statistical measurements to be organized in a certain way.
- division:
  - nominal scale,
  - ordinal scale,
  - interval scale,
  - ratio scale.

Nominal scale

- a scale in which a statistical unit is classified into a specific category.
- values in this scale have no ordering.
- example:

Religion	Code
Christianity	1
Islam	2
Buddhism	3

Ordinal scale

- values have a clearly defined order, but distances between them are not given,
- allows elements to be ranked.
- examples:

Education	Code
Primary	1
Secondary	2
Higher	3

Income	Code
Low	1
Medium	2
High	3

Interval scale

- feature values are expressed through specific numerical values,
- allows comparison of units (something is larger or smaller),
- it is not possible to examine ratios (determining how many times a given value is larger or smaller than another).
- example:

City	Temperature in $^\circ C$	Temperature in $^\circ F$
Warsaw	15	59
Olsztyn	10	50
Gdańsk	5	41
Szczecin	20	68

Ratio scale

- values are expressed through numerical values,
- it is possible to determine the smaller or larger relationship between values,
- it is possible to determine the ratio (quotient) between values,
- an absolute zero exists.
- example:

Product	Price in PLN
Bread	3
Butter	8
Pears	5

2.5 Types of statistical studies

- complete study - covers all units of the statistical population.
  - statistical census,
  - ongoing registration,

  - statistical reporting.

- partial studies - only a part of the population is observed. Conducted when a complete study is impractical or impossible.
  - monographic method,
  - representative method.

2.6 Stages of a statistical study

- designing and organizing the study: establishing the purpose, subject, object, scope, source, and duration of the study;
- statistical observation;
- processing statistical material: quality control of statistical material, grouping of obtained data, presentation of data results;
- statistical analysis.

2.7 Analysis of existing data

Analysis of existing data - the process of processing data in order to obtain useful information and conclusions from them. Depending on the type of data and the problems posed, this may involve the use of statistical, exploratory, and other methods.

Using existing data is an example of non-reactive research - methods of studying social behavior that do not influence those behaviors. Such data include: documents, archives, reports, chronicles, censuses, parish records, diaries, memoirs, internet blogs, audio memoirs, oral history archives, and others. (Wikipedia)

Existing data can be classified according to (Makowska ed. 2013):

- Nature: Quantitative, Qualitative
- Form: Processed data, Raw data
- Method of creation: Primary, Secondary
- Dynamics: Continuous event registration, Registration at time intervals, One-time registration
- Level of objectivity: Objective, Subjective
- Source of origin: Public data, Private data

Data analysis is a process of inspecting, organizing, transforming, and modeling data in order to obtain useful information, draw conclusions, and support the decision-making process. Data analysis has many aspects and approaches, encompassing various techniques under different names, in different business, scientific, and social domains. A practical approach to defining

data is that data are numbers, characters, images, or other recording methods, in a form that can be evaluated to determine or make a decision about a specific action. Many people believe that data in itself has no meaning - only processed and interpreted data becomes information.

2.8 Data analysis process

Analysis refers to breaking down the entirety of information into its distinct components for individual examination. Data analysis is the process of obtaining raw data and transforming it into information useful for decision-making by users. Data is collected and analyzed to answer questions, test hypotheses, or disprove theories. There are several phases that can be distinguished in the data analysis process. The phases are iterative, as feedback from subsequent phases may cause additional work in earlier phases.

2.8.1 Defining requirements

Before proceeding with data analysis, the quality requirements for data must be precisely defined. The input data to be analyzed are determined based on the requirements of those directing the analysis or clients (who will use the final product of the analysis). The general type of entity from which data will be collected is referred to as the experimental unit (e.g. a person or a population of people). Data can be numerical or categorical (i.e. text labels). The requirements definition phase should answer 2 fundamental questions:

- what do we want to measure?
- how do we want to measure it?

2.8.2 Data collection

Data is collected from various sources. Requirements regarding the type and quality of data may be communicated by analysts to "data custodians," such as information technology staff within an organization. Data may also be collected automatically from various types of sensors in the environment - such as traffic cameras, satellites, and devices recording images, sound, and physical parameters. Another method is obtaining data through interviews, gathering from online sources, or directly from documentation.

2.8.3 Data processing

Collected data must be processed or organized in a logical manner for analysis. For example, they may be placed in tables for further analysis - in a spreadsheet or other software. Data cleaning: After the processing and organizing phase, data may be incomplete, contain duplicates,

or contain errors. The need for data cleaning arises from problems related to data entry and storage. Data cleaning is the process of preventing and correcting detected errors. Common tasks include record matching, identifying inaccuracies, overall review of existing data quality, removing duplicates, and column segmentation. It is also extremely important to pay attention to data whose values are above or below previously established thresholds (extremes).

2.8.4 Proper data analysis

There are several methods that can be used for this purpose, for example data mining, business intelligence, data visualization, or exploratory research. The latter method is a way of analyzing datasets to determine their distinct characteristics. In this way, data can be used to test the original hypothesis. Descriptive statistics is another method for analyzing collected information. Data is examined to find its most important features. In descriptive statistics, analysts use several basic tools - the mean or average of a set of numbers can be used. This helps determine the overall trend, although it does not provide great accuracy when assessing the overall picture of collected data. In this phase, modeling and the creation of mathematical formulas also take place - they are applied to identify relationships between variables, such as correlation or causation.

2.8.5 Reporting and distribution of results

This phase involves determining in what form to communicate results. The analyst may consider various data visualization techniques to clearly and effectively convey the findings of the analysis to the audience. Data visualization uses graphical forms such as charts and tables. Tables are useful for users who can search for specific records, while charts (e.g. bar charts or line charts) provide a quantitative view of the analyzed dataset.

2.9 Where to get data?

Free data repositories:

- Local Data Bank of the Central Statistical Office (GUS) - link
- Open Data - link
- World Bank - link

2.10 "Tidy data" concept

The tidy data concept:

- WICKHAM, Hadley . Tidy Data. Journal of Statistical Software, [S.l.], v. 59, Issue 10, p. 1 - 23, sep. 2014. ISSN 1548-7660. Date accessed: 25 oct. 2018. doi:http://dx.doi.org/10.18637/jss.v059.i10.

2.10.1 "Tidy data" principles

Ideal data is presented in a table:

Name	Age	Height	Eye color
Adam	26	167	Brown
Sylwia	34	164	Hazel
Tomasz	42	183	Blue

What should we pay attention to?

- one observation (statistical unit) = one row in the table/matrix/data frame
- values of a given feature are found in columns
- one type/kind of observation in one table/matrix/data frame

2.10.2 Examples of untidy data

Name	Age	Height	Brown	Blue	Hazel
Adam	26	167	1	0	0
Sylwia	34	164	0	0	1
Tomasz	42	183	0	1	0

Column headers must correspond to features, not to variable values.

2.11 A few tips for good presentations

Edward Tufte, professor at Yale

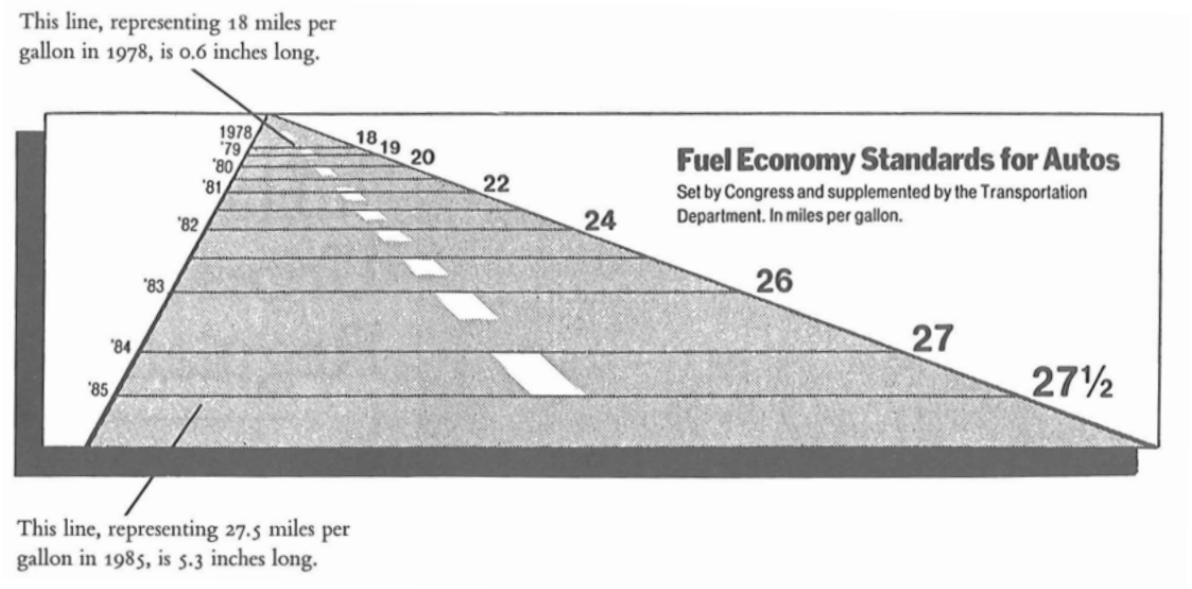
1. Present data richly.
2. Do not hide data, show the truth.
3. Do not use junk charts.
4. Show the variability of data, do not design it.

5. A chart should have the lowest possible lie factor.
6. PowerPoint is evil!

2.11.1 Lie factor

- the ratio of the effect visible on the chart to the effect indicated by the data on which the chart was drawn.

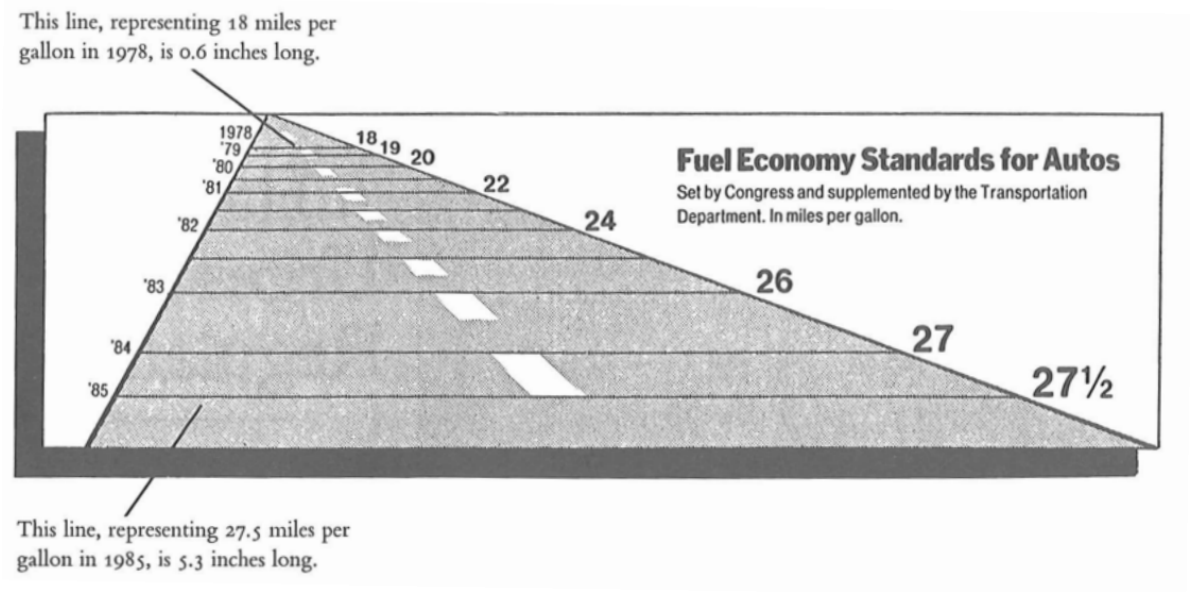
2.11.2 Lie factor



[Tufte, 1991] Edward Tufte, The Visual Display of Quantitative Information, Second Edition, Graphics Press, USA, 1991, p. 57 – 69.

$$\text{LieFactor} = \frac{\text{size of the effect visible on the chart}}{\text{size of the effect resulting from the data}}$$

$$\text{effect size} = \frac{|\text{second value} - \text{first value}|}{\text{first value}}$$



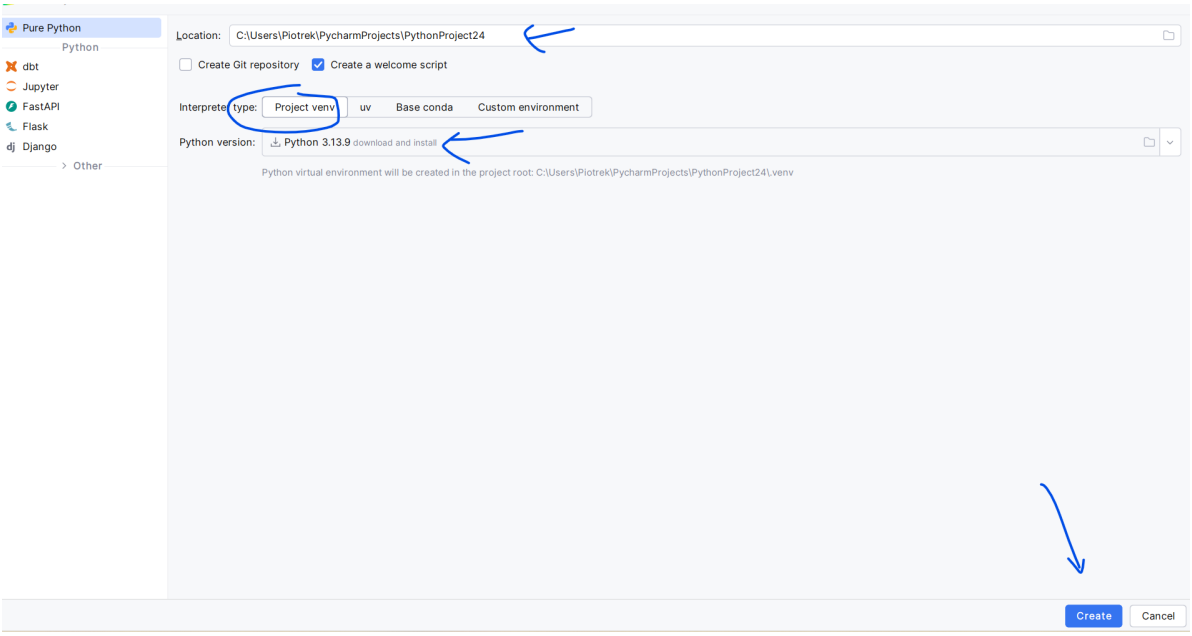
$$\text{LieFactor} = \frac{\frac{5.3-0.6}{0.6}}{\frac{27.5-18}{18}} \approx 14.8$$

2.12 References

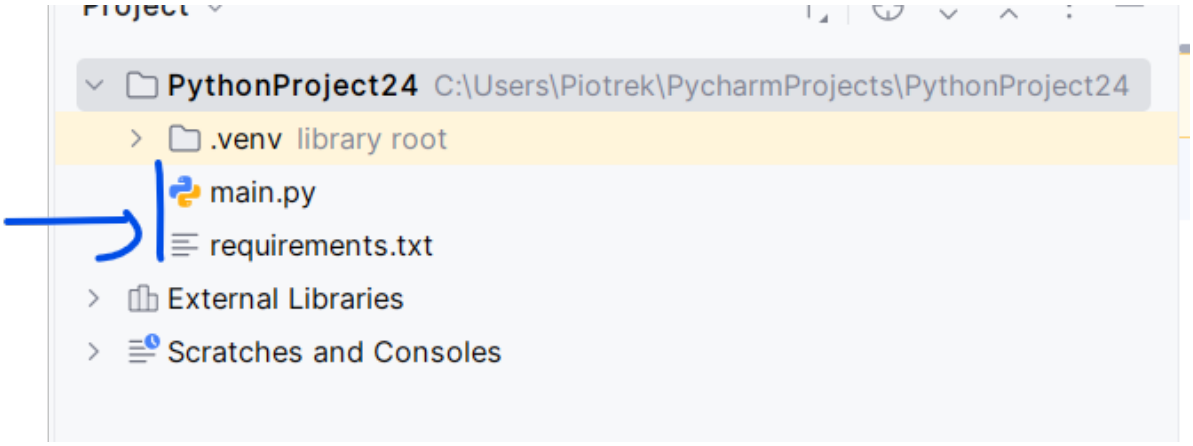
- https://pl.wikipedia.org/wiki/Wizualizacja
- https://mfiles.pl/pl/index.php/Analiza_danych, accessed online 1.04.2019.
- Walesiak M., Gatnar E., Statystyczna analiza danych z wykorzystaniem programu R, PWN, Warszawa, 2009.
- Wasilewska E., Statystyka opisowa od podstaw, Podręcznik z zadaniami, Wydawnictwo SGGW, Warszawa, 2009.
- https://en.wikipedia.org/wiki/Cognitive_reflection_test, accessed online 20.03.2023.
- https://qlikblog.pl/edward-tufte-dobre-praktyki-prezentacji-danych/, accessed online 20.03.2023.

3 Configuration

1. In the first step, we create a new project in the chosen location. We select venv and one of the Python versions, e.g. 3.13. Finally, click the Create button.



2. We add the requirements file https://piojas.pl/wp-content/uploads/2026/02/requirements.txt to the project's root directory. The file name is important.



3. We open the terminal in PyCharm and type the command: `pip install -r requirements.txt`

Part II

NumPy

4 NumPy - Getting Started

NumPy is a Python library for scientific computing.

Applications:

- linear algebra
- advanced mathematical (numerical) computations
- numerical integration
- solving equations

4.1 Importing the NumPy library

```
import numpy as np
```

The fundamental object in the NumPy library is an N-dimensional array called `ndarray`. Each element of the array is treated as a `dtype` type.

```
np.array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0, like=None)
```

- object - an object (or sequence) to be converted into an array
- dtype - type
- copy - whether objects should be copied, default is `True`
- order - memory layout: C (rows), F (columns), A, K
- subok - handled by subclasses (if `True`), default is `False`
- ndmin - minimum number of dimensions of the array
- like - creation based on a reference array

```
import numpy as np

a = np.array([1, 2, 3])                                    ①
print("a:", a)
print("type of a:", type(a))                               ②
b = np.array([1, 2, 3.0])                                  ③
```

```
print("b:", b)
c = np.array([[1, 2], [3, 4]])                                    ④
print("c:", c)
d = np.array([1, 2, 3], ndmin=2)                                  ⑤
print("d:", d)
e = np.array([1, 2, 3], dtype=complex)                            ⑥
print("e:", e)
f = np.array(np.asmatrix('1 2; 3 4'))                             ⑦
print("f:", f)
g = np.array(np.asmatrix('1 2; 3 4'), subok=True)                 ⑧
print("g:", g)
print(type(g))
```

① Creating an array with default settings.
② Checking the type.
③ One of the elements is of a different type. Therefore, upcasting to the "largest" type occurs.
④ Here we get a 2x2 array.
⑤ In this line we get an array with dimensions 1x3 (shape `(1, 3)`).
⑥ Forcing the `complex` type — a type with a wider range.
⑦ Using the matrix subtype.
⑧ Preserving the matrix type.

```
a: [1 2 3]
type of a: <class 'numpy.ndarray'>
b: [1. 2. 3.]
c: [[1 2]
 [3 4]]
d: [[1 2 3]]
e: [1.+0.j 2.+0.j 3.+0.j]
f: [[1 2]
 [3 4]]
g: [[1 2]
 [3 4]]
<class 'numpy.matrix'>
```

5 List vs Array

```
import numpy as np
import time

start_time = time.time()
my_arr = np.arange(1000000)
my_list = list(range(1000000))
start_time = time.time()
my_arr2 = my_arr * 2
print("--- %s seconds ---" % (time.time() - start_time))
start_time = time.time()
my_list2 = [x * 2 for x in my_list]
print("--- %s seconds ---" % (time.time() - start_time))
```

```
--- 0.0020012855529785156 seconds ---
--- 0.036900997161865234 seconds ---
```

6 Array Attributes

Attribute	Description
`shape`	tuple with information about the number of elements for each dimension
`size`	total number of elements in the array
`ndim`	number of dimensions of the array
`nbytes`	number of bytes the array occupies in memory
`dtype`	data type stored in the array (e.g. `int64`, `float64`)

```
import numpy as np

tab1 = np.array([2, -3, 4, -8, 1])
print("type:", type(tab1))
print("shape:", tab1.shape)
print("size:", tab1.size)
print("ndim:", tab1.ndim)
print("nbytes:", tab1.nbytes)
print("dtype:", tab1.dtype)
```

```
type: <class 'numpy.ndarray'>
shape: (5,)
size: 5
ndim: 1
nbytes: 40
dtype: int64
```

```
import numpy as np

tab2 = np.array([[2, -3], [4, -8]])
print("type:", type(tab2))
print("shape:", tab2.shape)
print("size:", tab2.size)
```

```
print("ndim:", tab2.ndim)
print("nbytes:", tab2.nbytes)
print("dtype:", tab2.dtype)
```

```
type: <class 'numpy.ndarray'>
shape: (2, 2)
size: 4
ndim: 2
nbytes: 32
dtype: int64
```

NumPy does not support ragged arrays. The following code will produce an error:

```
import numpy as np

tab3 = np.array([[2, -3], [4, -8, 5], [3]])
```

7 Data Types

Integer types	`int`,`int8`,`int16`,`int32`,`int64`
Unsigned integer types	`uint`,`uint8`,`uint16`,`uint32`,`uint64`
Boolean type	`bool`
Floating-point types	`float`, `float16`, `float32`, `float64`, `float128`
Complex floating-point types	`complex`, `complex64`, `complex128`, `complex256`
String	`str`

```
import numpy as np

tab = np.array([[2, -3], [4, -8]])
print(tab)
tab2 = np.array([[2, -3], [4, -8]], dtype=int)
print(tab2)
tab3 = np.array([[2, -3], [4, -8]], dtype=float)
print(tab3)
tab4 = np.array([[2, -3], [4, -8]], dtype=complex)
print(tab4)
```

```
[[ 2 -3]
 [ 4 -8]]
[[ 2 -3]
 [ 4 -8]]
[[ 2. -3.]
 [ 4. -8.]]
[[ 2.+0.j -3.+0.j]
 [ 4.+0.j -8.+0.j]]
```

8 Creating Arrays

`np.array` - arguments are cast to an array (anything iterable) - it's worth checking the size/shape

```
import numpy as np

tab = np.array([2, -3, 4])
print(tab)
print("size:", tab.size)
tab2 = np.array((4, -3, 3, 2))
print(tab2)
print("size:", tab2.size)
tab3 = np.array({3, 3, 2, 5, 2})
print(tab3)
print("size:", tab3.size)
tab4 = np.array({"pl": 344, "en": 22})
print(tab4)
print("size:", tab4.size)
```

```
[ 2 -3  4]
size: 3
[ 4 -3  3  2]
size: 4
{2, 3, 5}
size: 1
{'pl': 344, 'en': 22}
size: 1
```

`np.zeros` - creates an array filled with zeros

```
import numpy as np

tab = np.zeros(4)
print(tab)
print(tab.shape)
```

```
tab2 = np.zeros([2, 3])
print(tab2)
print(tab2.shape)
tab3 = np.zeros([2, 3, 4])
print(tab3)
print(tab3.shape)
```

```
[0. 0. 0. 0.]
(4,)
[[0. 0. 0.]
 [0. 0. 0.]]
(2, 3)
[[[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]

 [[0. 0. 0. 0.]
  [0. 0. 0. 0.]
  [0. 0. 0. 0.]]]
(2, 3, 4)
```

`np.ones` - creates an array filled with ones (this is not the equivalent of an identity matrix!)

```
import numpy as np

tab = np.ones(4)
print(tab)
print(tab.shape)
tab2 = np.ones([2, 3])
print(tab2)
print(tab2.shape)
tab3 = np.ones([2, 3, 4])
print(tab3)
print(tab3.shape)
```

```
[1. 1. 1. 1.]
(4,)
[[1. 1. 1.]
 [1. 1. 1.]]
(2, 3)
[[[1. 1. 1. 1.]
```

```
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]

 [[1. 1. 1. 1.]
  [1. 1. 1. 1.]
  [1. 1. 1. 1.]]]
(2, 3, 4)
```

`np.diag` - creates an array corresponding to a diagonal matrix

```
import numpy as np

print("tab0")
tab0 = np.diag([3, 4, 5])
print(tab0)
print("tab1")
tab1 = np.array([[2, 3, 4], [3, -4, 5], [3, 4, -5]])
print(tab1)
tab2 = np.diag(tab1)
print("tab2")
print(tab2)
tab3 = np.diag(tab1, k=1)
print("tab3")
print(tab3)
print("tab4")
tab4 = np.diag(tab1, k=-2)
print(tab4)
print("tab5")
tab5 = np.diag(np.diag(tab1))
print(tab5)
```

```
tab0
[[3 0 0]
 [0 4 0]
 [0 0 5]]
tab1
[[ 2  3  4]
 [ 3 -4  5]
 [ 3  4 -5]]
tab2
[ 2 -4 -5]
tab3
```

```
[3 5]
tab4
[3]
tab5
[[ 2  0  0]
 [ 0 -4  0]
 [ 0  0 -5]]
```

`np.arange` - array filled with evenly spaced values

Syntax: `numpy.arange([start, ]stop, [step, ]dtype=None)`

Works similarly to the `range` function, but allows fractional numbers.

```
import numpy as np

a = np.arange(3)
print(a)
b = np.arange(3.0)
print(b)
c = np.arange(3, 7)
print(c)
d = np.arange(3, 11, 2)
print(d)
e = np.arange(0, 1, 0.1)
print(e)
f = np.arange(3, 11, 2, dtype=float)
print(f)
g = np.arange(3, 10, 2)
print(g)
```

```
[0 1 2]
[0. 1. 2.]
[3 4 5 6]
[3 5 7 9]
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
[3. 5. 7. 9.]
[3 5 7 9]
```

`np.linspace` - array filled with evenly spaced values on a linear scale

`np.linspace(start, stop, num)` generates an array of evenly spaced values. For example, `np.linspace(2.0, 3.0, num=5)` produces `2, 2.25, 2.5, 2.75, 3` — the interval is divided

into `num - 1` equal parts. Setting `endpoint=False` removes the last point (from the right side), and the interval is then divided into `num` equal parts instead. The `dtype` parameter forces the output type — typically set to `int`, in which case the results may have the fractional part truncated.

```
import numpy as np

a = np.linspace(2.0, 3.0, num=5)
print(a)
b = np.linspace(2.0, 3.0, num=5, endpoint=False)
print(b)
c = np.linspace(10, 20, num=4)
print(c)
d = np.linspace(10, 20, num=4, dtype=int)
print(d)
```

```
[2.   2.25 2.5  2.75 3.  ]
[2.  2.2 2.4 2.6 2.8]
[10.         13.33333333 16.66666667 20.        ]
[10 13 16 20]
```

`np.logspace` - array filled with values on a logarithmic scale

Syntax: `numpy.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0)`

The function `np.logspace(2.0, 3.0, num=4)` generates 4 numbers spaced evenly on a logarithmic scale between 10^2 and 10^3. The default base is 10. First, the exponents are computed as equally spaced points in the interval [2.0, 3.0], which gives approximately 2, 2.33, 2.66, and 3.0. Then each exponent is used as the power of 10, producing the output values 10^2 = 100, 10^2.33 215.44, 10^2.66 464.16, and 10^3 = 1000. An important consequence is that while the exponents are uniformly distributed, the resulting values are not — the distances between consecutive output numbers grow larger as the values increase. This is the key distinction between `np.logspace` and `np.linspace`: the former creates equal spacing in log-scale (i.e. between the exponents), not in the actual output values.

```
import numpy as np

a = np.logspace(2.0, 3.0, num=4)
print(a)
b = np.logspace(2.0, 3.0, num=4, endpoint=False)
print(b)
```

```
c = np.logspace(2.0, 3.0, num=4, base=2.0)
print(c)
```

```
[ 100.          215.443469    464.15888336 1000.        ]
[100.         177.827941   316.22776602 562.34132519]
[4.         5.0396842  6.34960421 8.        ]
```

`np.empty` - empty (uninitialized) array - specific values are not "guaranteed"

```
import numpy as np

a = np.empty(3)
print(a)
b = np.empty(3, dtype=int)
print(b)
```

```
[0. 1. 2.]
[                  0 4607182418800017408 4611686018427387904]
```

`np.identity` - array resembling an identity matrix

`np.eye` - array with ones on the diagonal (remaining values are zeros)

```
import numpy as np

print("a")
a = np.identity(4)
print(a)
print("b")
b = np.eye(4, k=1)
print(b)
print("c")
c = np.eye(4, k=2)
print(c)
print("d")
d = np.eye(4, k=-1)
print(d)
```

```
a
[[1. 0. 0. 0.]
```

```
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
b
[[0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]
 [0. 0. 0. 0.]]
c
[[0. 0. 1. 0.]
 [0. 0. 0. 1.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
d
[[0. 0. 0. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]]
```